

IndraMotion MLC IndraLogic XLC

Exercises

MLC/XLC Version 12



This document is part of the training documentation of the seminar IndraMotion MLC / IndraLogic XLC.

It is not designed for self-study. For a complete understanding the explanations during the seminar are required.

All information in product documentation, manuals and other publications by Bosch Rexroth AG for the used products has to be considered.

© Bosch Rexroth AG 2015. All rights reserved, also regarding any disposal, exploitation, reproduction, editing, distribution, as well as in the event of applications for industrial property rights.

Contents

1	About this document	4
2	Loading Base Parameters	5
3	Using the Control Display	6
4	Network Settings	7
5	Firmware Management	8
6	Create IndraWorks Project	9
7	IndraDrive Firmware Download	10
8	Motion Programming with PLCopen	11
9	Absolute Positioning with PLCopen	13
10	Diagnosis	14
11	Use of the Axis Interface	17
12	Single Axis Movements with Axis Interface	23
13	Adding Diagnosis	26
14	Jogging with AxisInterface	27
15	Extension of the AxisInterface	28
16	Parameter Import and Export	31
17	Archiving and Restoring Projects	33
18	Velocity Synchronization	34
19	Phase Synchronization	35
20	Create Drive CAM with CamBuilder	36
21	Electronic CAM (drive CAM)	37
22	Motion Profile	38
23	Create FlexProfile with CamBuilder	40
24	FlexProfile	41
25	Online Creation of FlexProfiles	43
26	FlexProfile and Events	45
27	Notes	48

1 About this document

1.1 Validity of this document

This documentation is intended for participants of the seminar IndraLogic XLC / IndraMotion MLC. It contains practical exercises which are part of this seminar.

1.2 Required and Supplementing Documentation



The complete product documentation is available on the Media Directory (www.boschrexroth.com/mediadirectory). The installation DVD contains a subfolder "Documentation" with all manuals for the current system version. The complete information of the product manuals is integrated equally in the Online Help system.

► Please refer to these documents in case of questions.

1.3 Symbols

The following symbols are used for an ease of understanding:

Symbol	Meaning
	Tip, additional information or reference to other sources of information.
►	Single, independent instruction
1.	Numbered list of instructions:
2.	The digits indicate that there is a sequence of consecutive steps which have to
3.	be executed in the correct order.

2 Loading base parameters

From previous usage, the control can contain old program components, parameterizations and PLC data that have to be removed by clearing the memory.

All parameters of the MLC are reset to default values, i.e. the delivery state is restored (exception: IP parameters).

With brand-new controls, this step can be omitted.

2.1 Exercise

- Set the MLC back to delivery state by loading base parameters.



The procedure how to load base parameters is described in the presentation. Please refer to the printout if you need more information!

3 Using the control display

The control display is used to display the control status, configure the network settings and trigger single commands.

The status of the control is displayed after startup. If there is no error, the status of the PLC (STOP or RUN) and the status of the SERCOS (P0, P2 or BB) are displayed. If there is an error, the error number is alternately shown with a brief error description. Press "ESC" on the display to acknowledge the indicated error.



The menu structure is described in the presentation. Please refer to the printout if you need more information!

3.1 Exercise

1. Press "Enter" and select the menu "Device Info".
2. Please retrieve the following information:

Installed Firmware:

Type code:

Hardware Index:

Serial number:

OHC fan:

OHC CPU:

Temperature:

4 Network settings

The network settings of the MLC also are configured by the means of the control display. In this exercise the networks settings of the MLC are adjusted and the Ethernet connection to the notebook is checked.

4.1 Exercise

1. Adjust the MLC network settings as follows:

IP address:	192.168.123. 141
Subnet mask:	255.255.255.0
Gateway:	192.168.123.254 (IP address of notebook)



If IP parameters of the MLC are changed, the modifications are effective only after a restart!

2. Restart the MLC (switch off and on).
3. Check the IP parameters of the engineering notebook and adjust them if necessary:

IP address:	192.168.123.254 (Address on Laptop)
Subnet mask:	255.255.255.0

4. Connect the notebook with the engineering port of the MLC.
5. Open a DOS box and use the standard windows command ping to test the Ethernet connection:

Ping 192.168.123.141

5 Firmware management

In this exercise the firmware of the MLC is downloaded to ensure that the correct firmware is installed on the control.



A detailed description of the firmware download procedure is included in the presentation. If you need more information please refer to the printout!

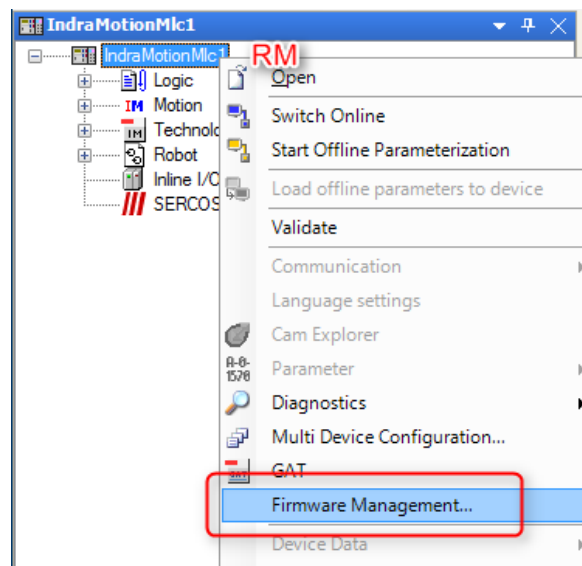
5.1 Exercise

1. Create an empty IndraWorks project.
2. Add an MLC to the project and enter the correct IP address for the MLC.



In the first step only the control is added to the project. More components like drives, virtual axes, IOs etc. will be added later. For the firmware download only the MLC is required!

3. Call the menu entry "Firmware Management":



The control generally reboots automatically. However, in some cases the control has to be restarted manually. In this case, "PWRCYCLE" is displayed on the control display. Disconnect the supply voltage for a short period and then restart the control.

6 Create an IndraWorks project

In this exercise the previously created IndraWorks project is completed. Drives on the SERCOS interface can be scanned automatically; other SERCOS nodes need to be added manually to the project as well as local Inline IOs, PROFIBUS IOs etc.

6.1 Exercise

1. Scan SERCOS bus configuration.
2. Use the remote address assignment to setup the correct SERCOS addresses:

SERCOS address of left IndraDrive:**11**

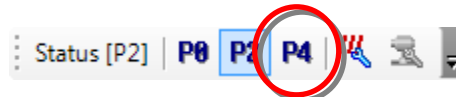
SERCOS address of right IndraDrive:**12**

3. Please enter the name “Axis1” for the IndraDrive with address 11 and “vAxis2” for the IndraDrive with address 12 and select “Interpolation on Drive”.
4. Add the drives to the project.
5. Select the Inline I/O's mounted locally on the MLC an add them in the project.
6. After adding all hardware components to the project switch online.



After switching online an Ethernet connection is established and the projected devices are compared to the connected devices. If there is a mismatch the differences are displayed and the problems have to be fixed before continuing!

7. Switch the SERCOS from P2 to BB and watch the control display as well as the drive display and the LEDs on the SERCOS bus coupler.



8. Errors are displayed on the control display as well as in IndraWorks. The complete list of all errors, warnings and info's is available in IndraWorks. Please list the active errors:



.....

.....

.....

.....

.....

.....

.....

7 IndraDrive firmware download

In this exercise the Firmware of the IndraDrive is downloaded.



For firmware download on regular IndraDrive and for IndraDrive and IndraDrive Mi a MMC card can be used. On the IndraDrive Cs this option is not available; the firmware download has to be achieved via Ethernet.



A detailed description of the firmware download for IndraDrive Cs is included in the presentation. Please follow the instructions in the printout!

7.1 Exercise

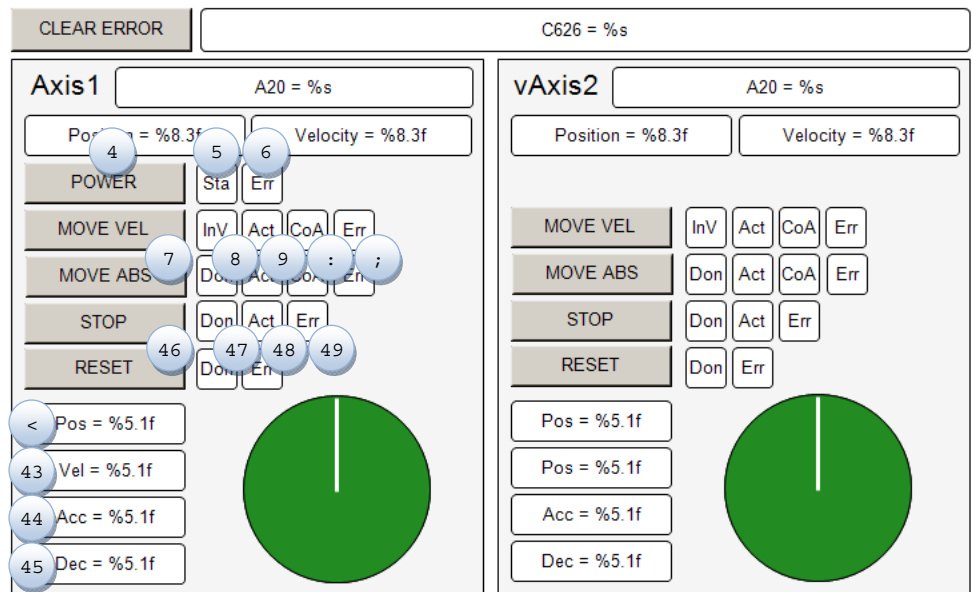
- Upgrade or downgrade the firmware of the IndraDrive. The required firmware file is on the MMC / engineering notebook.

8 Motion programming with PLCopen

During this exercise single axis movements are programmed by means of standardized PLCopen function blocks. To do this exercise you need an import file with a ready-made IndraLogic visualization.

8.1 Visualization

The following visualization is used for this exercise:

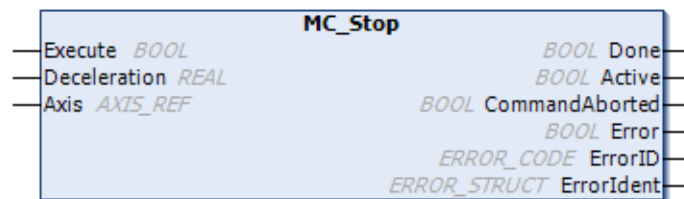
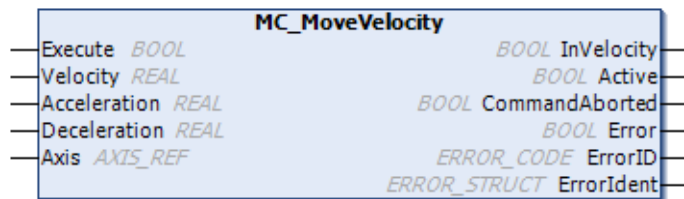
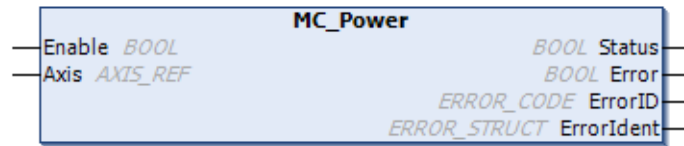


Element #	Assigned Variable	Element #	Assigned Variable
1	bEnablePower_Axis1	9	rPos_Axis1
2	bStatusPower_Axis1	10	rVel_Axis1
3	bErrPower_Axis1	11	rAcc_Axis1
4	bExeMoveVelocity_Axis1	12	rDec_Axis1
5	bInVel_MoveVelocity_Axis1	13	bExe_Stop_Axis1
6	bAct_MoveVelocity_Axis1	14	bDone_Stop_Axis1
7	bAborted_MoveVelocity_Axis1	15	bAct_Stop_Axis1
8	bErr_MoveVelocity_Axis1	16	bErr_Stop_Axis1

An equal set of variables exists for vAxis2. For the names of the assigned variables just replace “_Axis1” by “_vAxis2”, e.g. rPos_Axis1 is used for Axis1 and rPos_vAxis2 is used for axis vAxis2.

8.2 Exercise

9. To use the described IndraLogic visualization import the file „Visu_PlcOpen.iwx“!
10. Open the program PlcProg and add the required PLCopen function blocks!
11. Use the following three function blocks for the real axis:



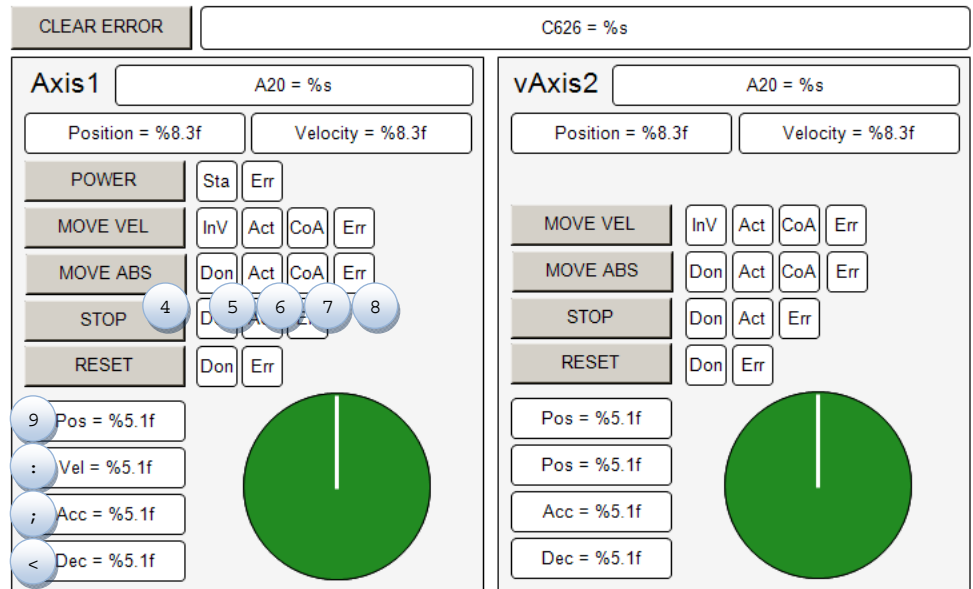
12. After implementing such single axis operations for Axis1, implement the same functions for vAxis2. For a virtual axis the MC_Power function block is not required.
13. The proper working of the axes can be monitored in IndraWorks.

9 Absolute positioning with PLCopen

In this exercise absolute positioning is added to the previous exercise.

9.1 Visualization

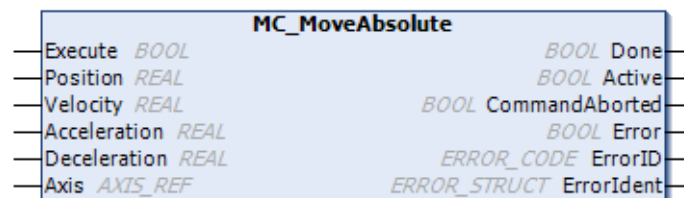
The visualization of the previous exercise is used also for this exercise:



Element #	Assigned Variable	Element #	Assigned Variable
1	bExe_MC_MoveAbsolute_Axis1	6	rPos_Axis1
2	bDone_MoveAbsolute_Axis1	7	rVel_Axis1
3	bAct_MoveAbsolute_Axis1	8	rAcc_Axis1
4	bAborted_MoveAbsolute_Axis1	9	rDec_Axis1
5	bErr_MoveAbsolute_Axis1		

9.2 Exercise

1. Open the program and add an instance of MC_MoveAbsolute:



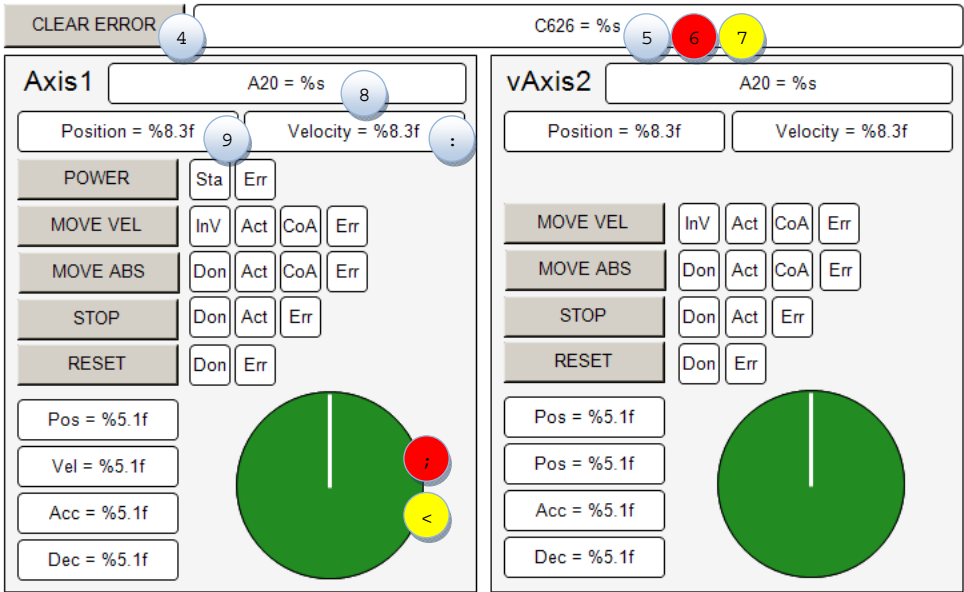
2. Create a second instance of this function block for vAxis2.
3. The proper working of the axes can be monitored in IndraWorks.

10
Diagnosis

This exercise demonstrates how to determine the status of your MLC/XLC and the status of the axes from within the motion program. A complete set of function blocks is available to read any of the control or drive parameters. In addition there are global variables (AxisData, DV_Axis, DV_Control) to provide such diagnosis information.

10.1
Visualization

The visualization of the previous exercise is used also for this exercise:



Element #	Assigned Variable	Element #	Assigned Variable
1	bExe_ClearError	6	rActualPosition_Axis1
2	strMLCInfo	7	rActualVelocity_Axis1
3	bMLCError (red color)	8	bError_Axis1 (red color)
4	bMLCWarning (yellow color)	9	bWarning_Axis1 (yellow color)
5	strErrorMessage_Axis1		

10.2 Exercise

1. Which parameters are available for diagnosis purposes?

C-Parameters:

MLC system status:

MLC diagnostic message (string):

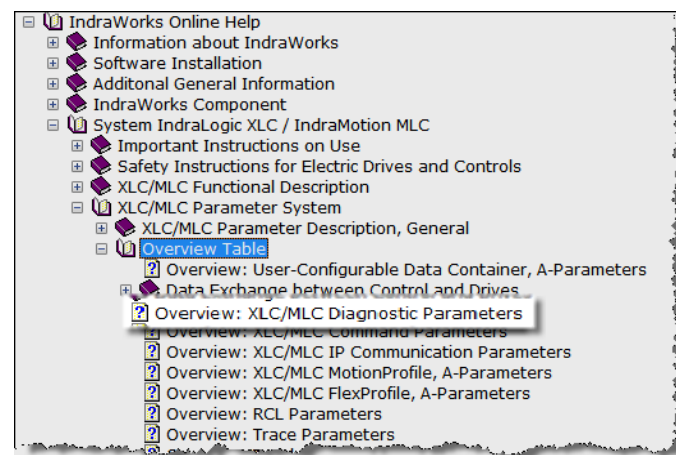
A-Parameters:

Axis status:

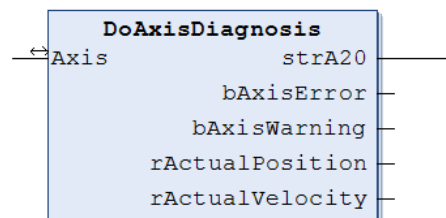
Axis diagnostic message (string):



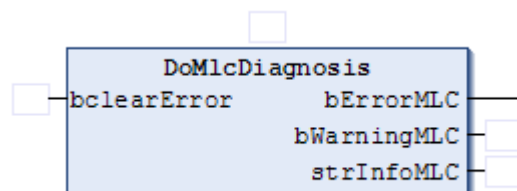
To get an overview on all parameters relating to diagnosis, the Online Help in IndraWorks can be used:



2. Implement the following function block for axis related diagnosis information:



3. Implement a function block for MLC-related diagnosis information:



4. Open the program PlcProg and add an instance of DoMlcDiagnosis and also one instance of DoAxisDiagnosis per axis.

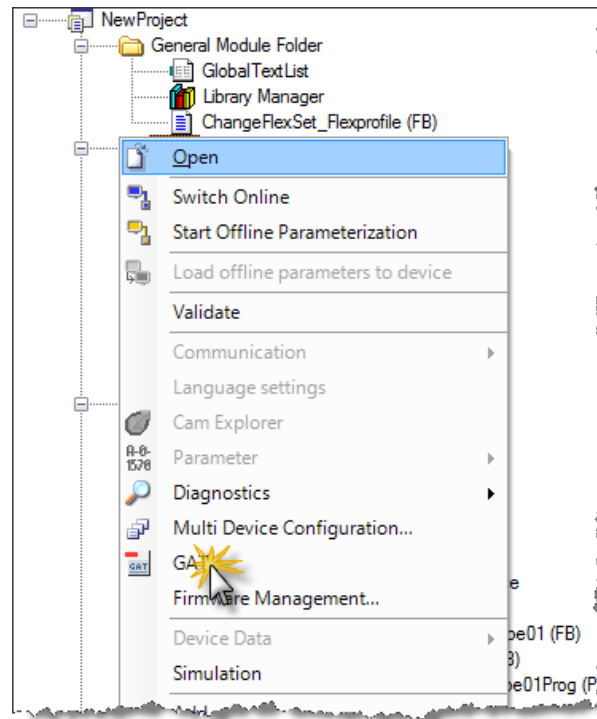
5. After compiling your modified motion program and downloading it to the MLC the diagnosis information is displayed in the IndraLogic visualization. Check if the displayed information is OK!

11 Use of the Axis Interface

With the Axis Interface Rexroth provides an easier way for motion programming. In this exercise the Axis Interface is added to the project and a ready-made visualization which is part of the Axis Interface is used to move the axes.

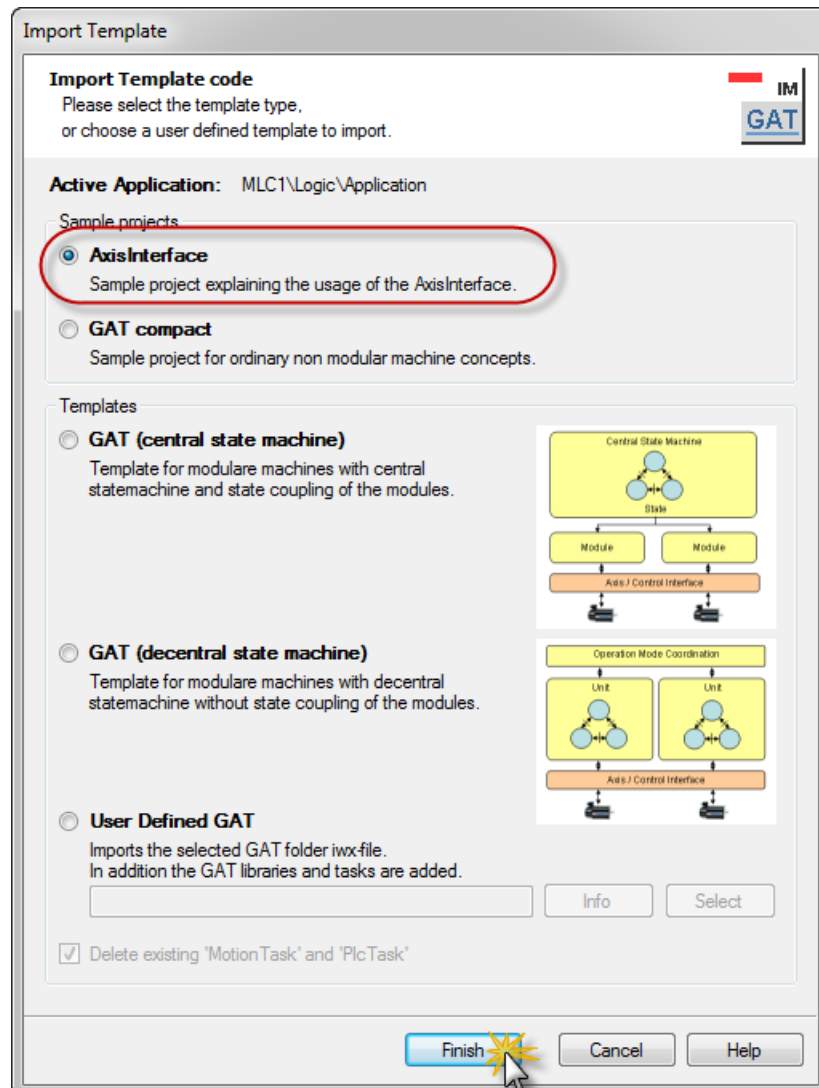
11.1 Including the Axis Interface to the project

1. Call the GAT assistant to add the Axis Interface to your project:



The project structure is modified by the GAT assistant. Please switch offline and logout from the PLC before calling the GAT assistant!

2. Select “AxisInterface” and press “Finish”:



As a result the Axis Interface is added to your project and ready to use!



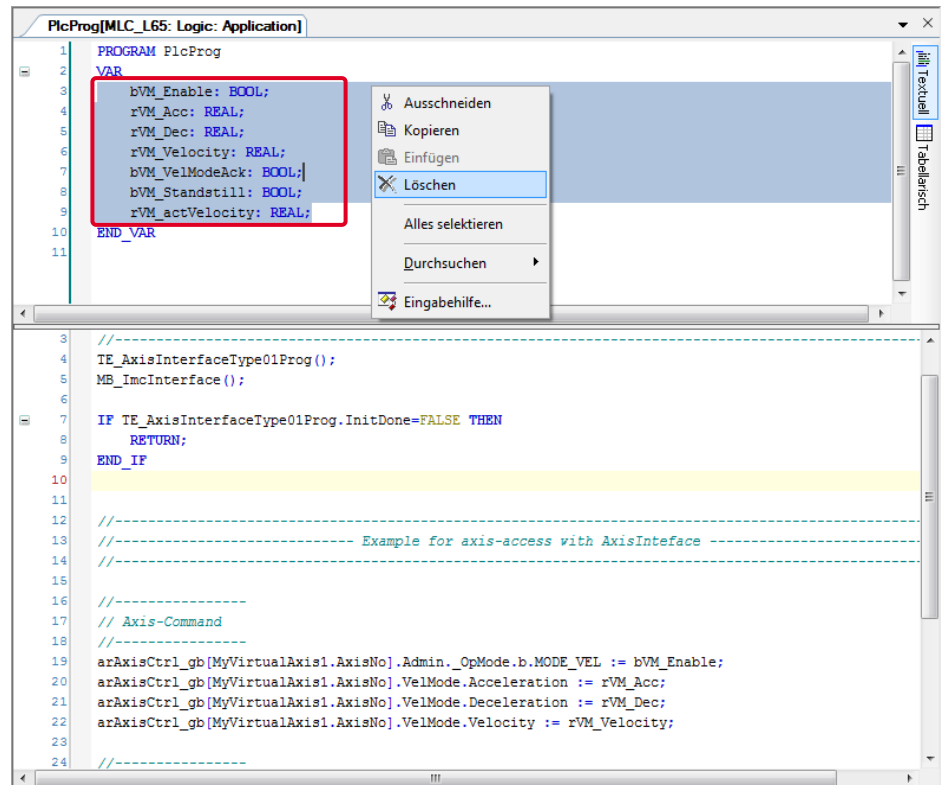
The assistant modifies the task configuration and also adds two virtual axes to your project!

3. Delete 2 new virtual axes from your project! Open the folder “Virtual Axes” and delete “MyVirtualAxis1” and “MyVirtualAxis2”!

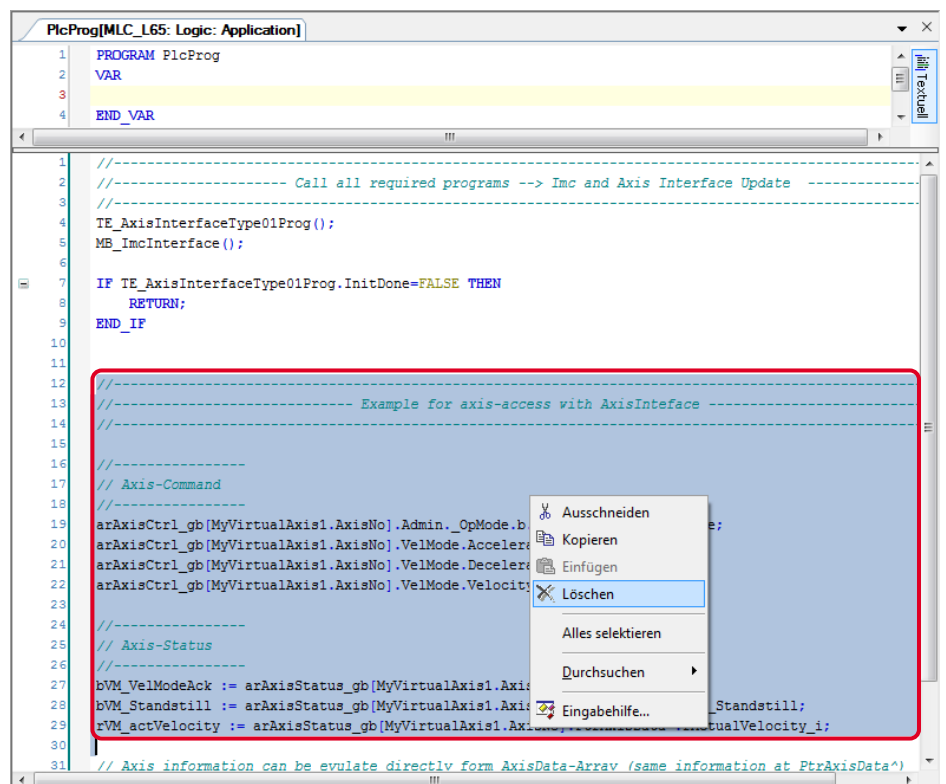


The assistant also inserts a new program PlcProg and a new folder UserVarGlobal to your project. For both items a backup has been created: PlcProg_Backup and UserVarGlobal_Backup!

4. Open PlcProg and delete the unnecessary declarations, which have been added by the assistant:



5. Delete also the unnecessary statements in PlcProg:





The starting lines in PlcProg ensure the proper working of the Axis Interface. Never delete these instructions from PlcProg and don't modify them.

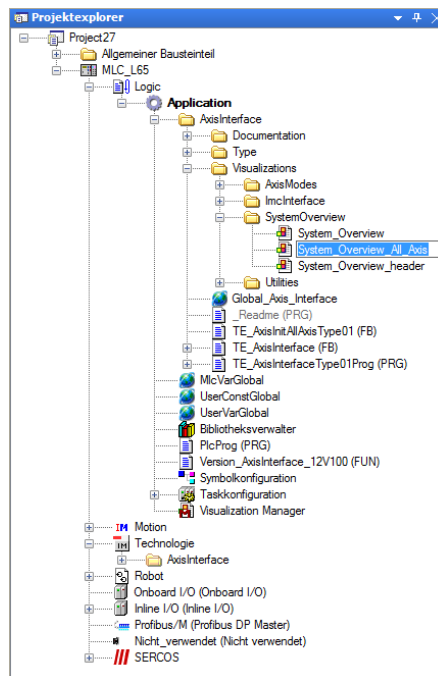
```

PlcProg[MLC_L65: Logic: Application]
1  PROGRAM PlcProg
2  VAR
3
4  END_VAR

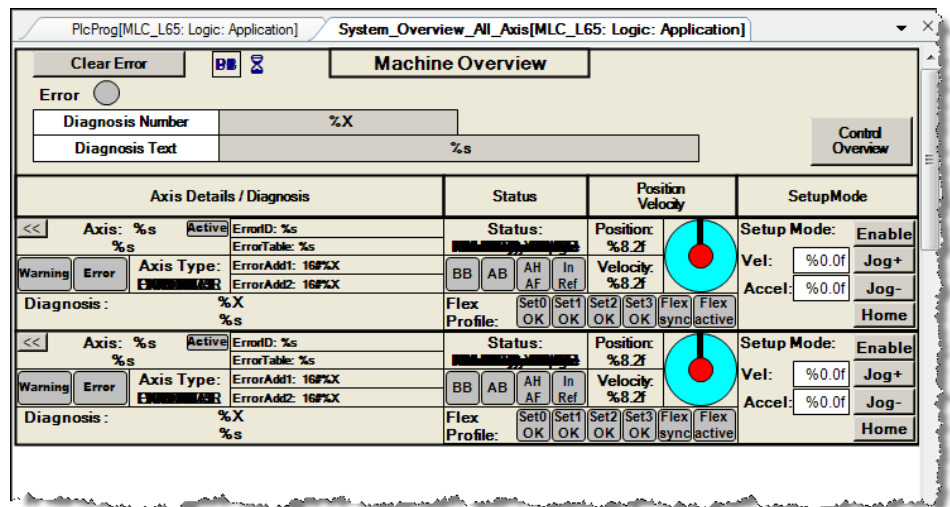
1  //----- Call all required programs --> Imc and Axis Interface Update -----
2  //-----
3  TE_AxisInterfaceType01Prog();
4  MB_ImcInterface();
5
6
7  IF TE_AxisInterfaceType01Prog.InitDone=FALSE THEN
8      RETURN;
9  END_IF
10

```

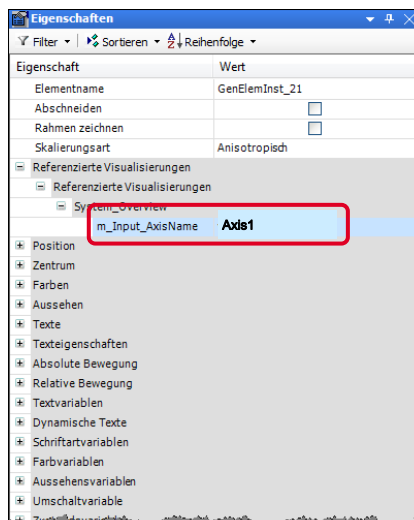
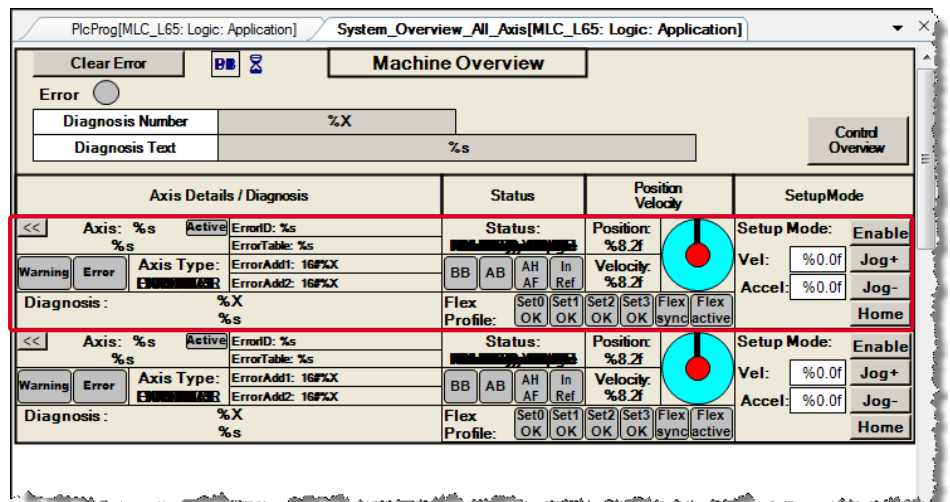
6. Open the IndraLogic visualization “System_Overview_All_Axis”:



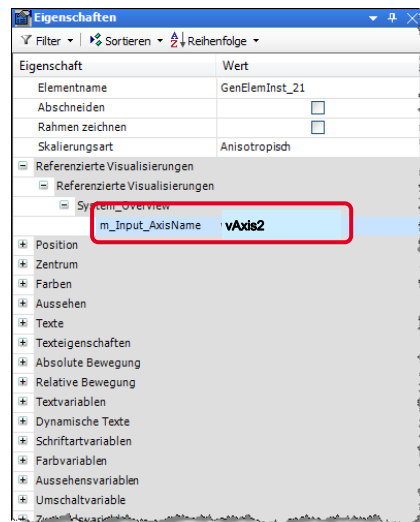
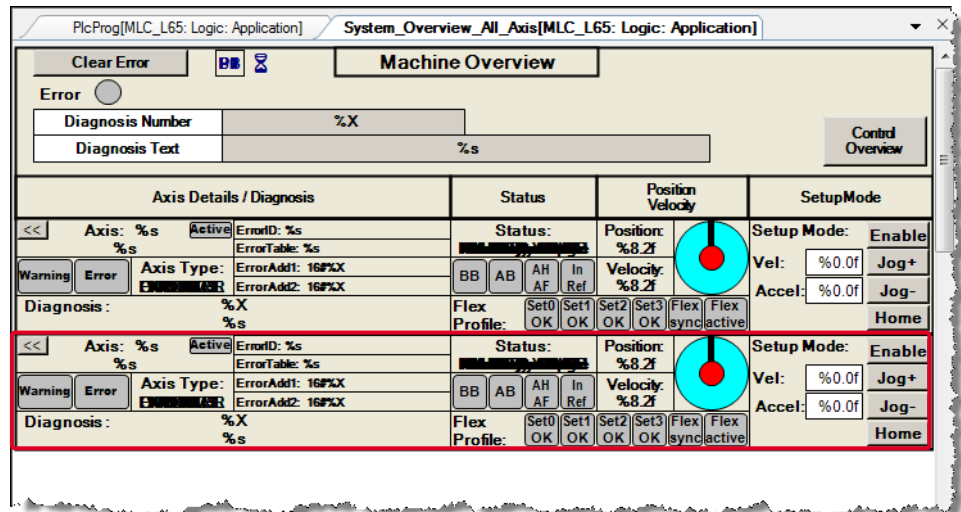
7. Open the IndraLogic visualization “System_Overview_All_Axis”:



8. Select the visualization frame for the 1st axis by a mouse click and change the reference to “Axis1”.



9. Select the visualization frame for the 2nd axis by a mouse click and change the reference to "vAxis2".



10. After going online with IndraWorks, compiling the IndraLogic program and logging in you can use the Visualization to control the axes.
11. Check the proper working of the Axis Interface!

12 Single axis movements with Axis Interface

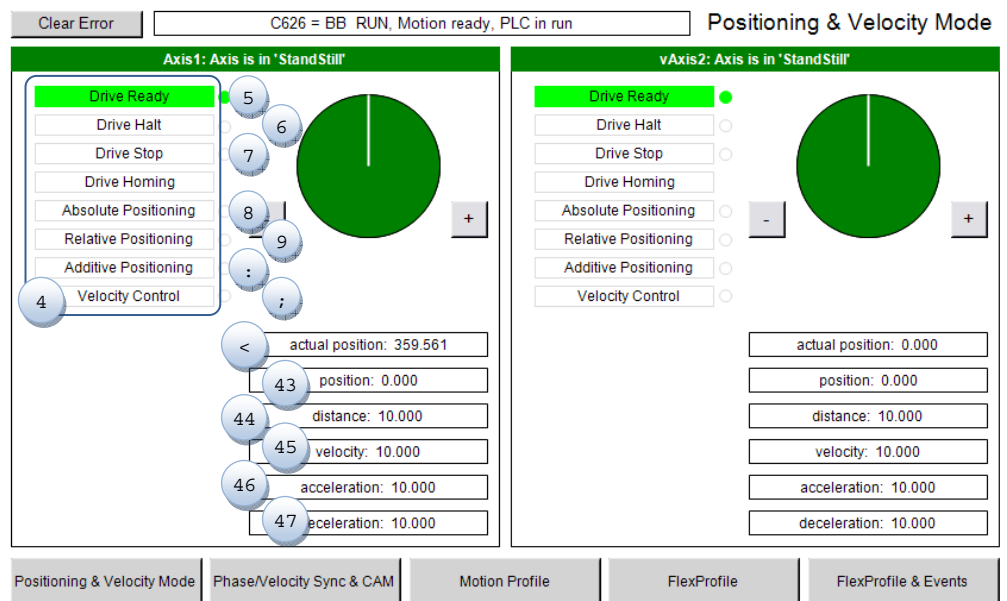
In this exercise single axis movements are programmed based on the AxisInterface.

12.1 Visualization



For the following exercise the standard IndraLogic visualization coming with the Axis Interface is no longer used. This commissioning visualization can be used at the beginning before programming in IndraLogic. Afterwards you can't use it anymore, because the settings via the visualization are overwritten by the statements in the motion program.

The following new visualization is used as a basis for this exercise:



Element #	Assigned Variable	Element #	Assigned Variable
1	iSelOpMode_Axis1	8	bInVel_Axis1
2	bInAb_Axis1	9	rActualPosition_Axis1
3	bInAH_Axis1	10	rPosition_Axis1
4	bStopping_Axis1	11	rDistance_Axis1
5	bInPos_Axis1	12	rVelocity_Axis1
6	bInPosRel_Axis1	13	rAcceleration_Axis1
7	bInPosAdd_Axis1	14	rDeceleration_Axis1

An equal set of variables exists for vAxis2. For the names of the assigned variables just replace “_Axis1” by “_vAxis2”, e.g. bInAH_Axis1 is used for the real axis Axis1 and bInAH_vAxis2 is used for axis vAxis2.

12.2 Exercise

1. To use the described IndraLogic visualization import the file "Visu_AxisInterface.iwx"!
2. The operating mode which is selected in the visualization is reported by the variable iSelOpmode_Axis1. The following table describes the connection between the selected operating mode and the value of this variable:

iSelOpmode_Axis 1	Operating Mode
0	Drive Ready
1	Drive Halt
2	Drive Stop
3	Drive Homing
4	Absolute Positioning
5	Relative Positioning
6	Additive Positioning
7	Velocity Control

In the first step implement the operating modes printed in bold characters. Use a CASE statement for this purpose:

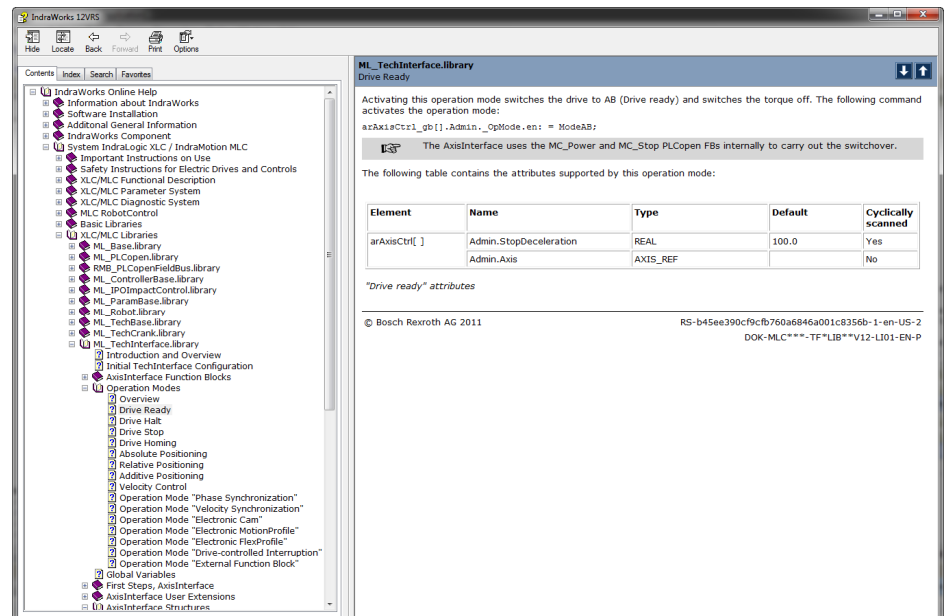
```

CASE iSelOpmode_Axis1 OF
0: // Drive Ready
1: // Drive Halt
2: // Drive Stop
3: // Drive Homing
4: // Absolute Positioning
   arAxisCtrl_gb[Axis1.AxisNo].Admin._OpMode.en := ModePosAbs;
   arAxisCtrl_gb[Axis1.AxisNo].PosMode.Position := rPosition_Axis1;
   arAxisCtrl_gb[Axis1.AxisNo].PosMode.Velocity := rVelocity_Axis1;
   arAxisCtrl_gb[Axis1.AxisNo].PosMode.Acceleration := ...
   ...
5: // Relative Positioning
6: // Additive Positioning
7: // Velocity Control
8: // Phase Synchronization
9: // Velocity Synchronization
10: // Electronic CAM
11: // Electronic MotionProfile
12: // Electronic FlexProfile
END_CASE;

```



A complete description of the Axis Interface is available in the Online Help. If you want to implement a specific operating mode you can always refer to the Online Help!



- After implementation of the different functions for the 1st axis, copy the code to have the same functions also for vAxis2.
- The proper working of the axes can be monitored in IndraWorks.
- All information which is exchanged via the AxisInterface can be monitored in IndraWorks. The node Technology/AxisInterface/Axis1 displays the information for Axis1. For vAxis2 there is a corresponding node Technology/AxisInterface/vAxis2. Check your settings via the Axis Interface.

13 Adding diagnosis

For the further exercises it is helpful to display MLC and axis status information. Implement the corresponding functions. The code of the previous diagnosis exercise can be imported. You don't need to implement these functions twice!

13.1 Visualization

The visualization of the previous exercise is used also for this exercise:



Element #	Assigned Variable	Element #	Assigned Variable
1	bExe_ClearError	5	strErrorMessage_Axis1
2	strMLCInfo	6	bError_Axis1
3	bMLC_Error	7	bWarning_Axis1
4	bMLC_Warning	8	rActualPosition_Axis1

The variables used for Axis1 also exist for vAxis2. For the names of the assigned variables just replace “_Axis1” by “_vAxis2”, e.g. bError_Axis1 is used for Axis1 and bError_vAxis2 is used for vAxis2.

13.2 Exercise

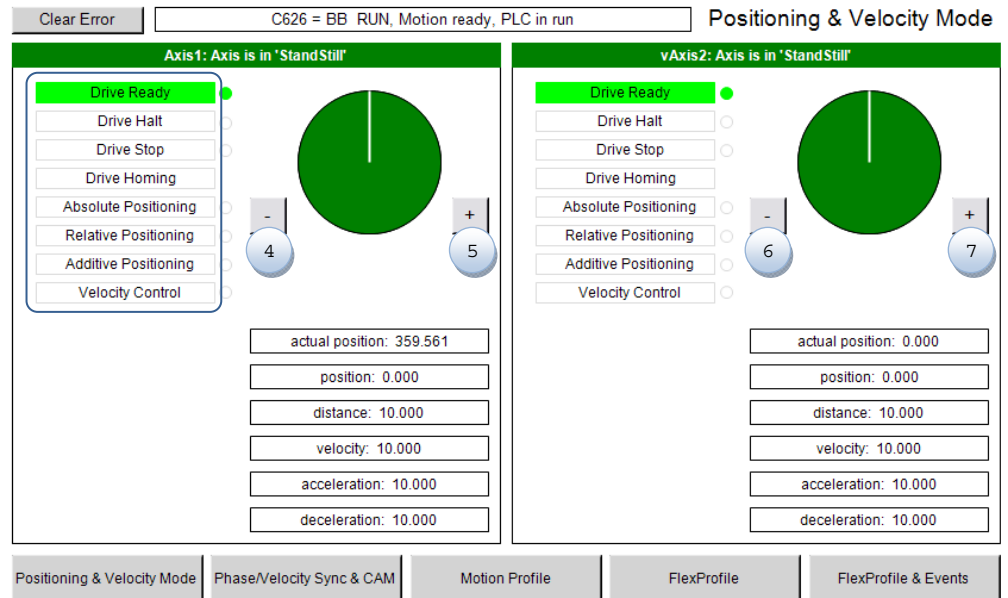
- Add the code from the previous diagnosis exercise to your IndraLogic project. Either use cut & paste or the export / import function.

14 Jogging with AxisInterface

In this exercise jogging is implemented for both axes.

14.1 Visualization

The visualization of the previous exercise is used also for this exercise:



Element #	Assigned Variable	Element #	Assigned Variable
1	bJogMinus_Axis1	5	bJogMinus_vAxis2
2	bJogPlus_Axis1	6	bJogPlus_vAxis2

14.2 Exercise

- Implement Jogging for both axes using the Axis Interface. Extend your program as follows:

```

IF bJogMinus_Axis1 OR bJogPlus_Axis1 THEN
  // Jogging
  arAxisCtrl_gb[Axis1.AxisNo].SetupMode.Enable := TRUE;
  ...
ELSE
  // regular operating mode
  arAxisCtrl_gb[Axis1.AxisNo].SetupMode.Enable := FALSE;
  CASE iSelOpmode_Axis1 OF
    0: // Drive Ready
    ...
  END_CASE;
END_IF;

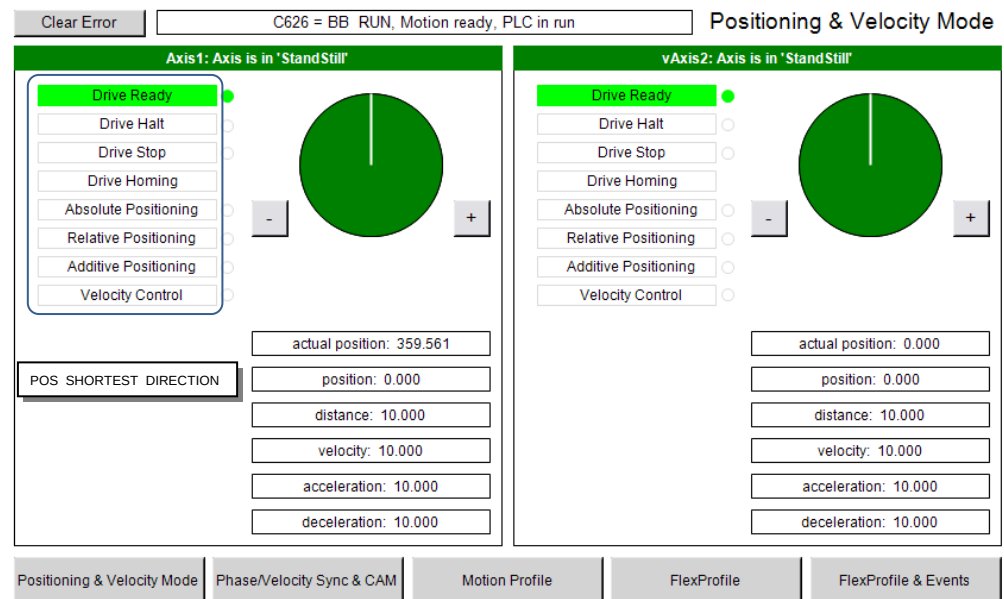
```

15 Extension of the AxisInterface

Sometimes an extension of the AxisInterface is useful. For absolute positioning of a rotary axis there is a parameter A-0-0203 which specifies, if the commanded axis moves in positive direction, in negative direction, or if it moves to the target position at the shortest distance. This parameter is preset normally. If you want to indicate the direction of the motion with every positioning command, an extension of the AxisInterface is advisable.

15.1 Visualization

The visualization of the previous exercise will be used but it has to be extended:



15.2 Exercise

1. Add type EN_DIRECTION_POSABS to Logic/Application/AxisInterface/Control:

TYPE EN_DIRECTION_POSABS :

```
(
  POS_SHORTEST_DIRECTION := 0,
  POS_POSITIVE_DIRECTION := 1,
  POS_NEGATIVE_DIRECTION := 2
);
```

END_TYPE

2. A new component has to be added to the type TE_AXIS_POSITIONING in Logic/Application/AxisInterface/Control:

(* add your extensions below here, keep the elements above
Erweiterungen hier einfügen, Elemente oberhalb ... *)

```
enDirectionPosAbs : EN_DIRECTION_POSABS :=
  POS_SHORTEST_DIRECTION;
N;
```

3. The setting inside the AxisInterface has to be copied to parameter A-0-0203. For this reason add a new action actPosModeDirection to the function block TE_AxisInterface, which contains one line of code:

```
DV_Axis[AdminCtrlExt.Axis.AxisNo].A_0_0203 :=
    PosModeCtrlExt.enDirectionPosAbs;
```

4. Call this action inside the function block TE_AxisInterface:

```
(* add your extensions below here, keep the elements above
Erweiterungen hier einfuegen, Elemente oberhalb nicht ...
(* user extensions | Anwender Erweiterungen *)
actPosModeDirection();
```

5. Open the folder Logic/Application/Visu_AxisInterface/Visu_VarGlobal and add 2 new global variables to this folder:

```
enPosAbsDir: EN_DIRECTION_POSABS :=
    POS_SHORTEST_DIRECTION;

bTogglePosAbsDir : BOOL;
```

6. Add a new element to the visualization VisuPosVelMode as shown above. Set the properties as follows:

Texts/Text: %s

Text variables: enPosAbsDir

Input configuration / Toggle: bTogglePosAbsDir

POS SHORTEST DIRECTION

7. Create a new function block to be able to modify enPosAbsDir by a mouse click:

FUNCTION_BLOCK TogglePosAbsDirFB

VAR_INPUT

END_VAR

VAR_OUTPUT

END_VAR

VAR

bTogglePosAbsDirPrev: **BOOL**;

END_VAR

IF bTogglePosAbsDir <> bTogglePosAbsDirPrev **THEN**

bTogglePosAbsDirPrev := bTogglePosAbsDir;

CASE enPosAbsDir **OF**

POS_SHORTEST_DIRECTION: enPosAbsDir :=

POS_POSITIVE_DIRECTION;

POS_POSITIVE_DIRECTION: enPosAbsDir :=

POS_NEGATIVE_DIRECTION;

ELSE enPosAbsDir := POS_SHORTEST_DIRECTION;

END_CASE

END_IF

8. Create an instance of this function block and call it cyclically.
9. The AxisInterface holds now a new component which can be used for every axis:

4:// Absolute Positioning

arAxisCtrl_gb[Axis1.AxisNo].Admin._OpMode.en := ModePosAbs;

arAxisCtrl_gb[Axis1.AxisNo].PosMode.Position :=

rPosition_Axis1;

arAxisCtrl_gb[Axis1.AxisNo].PosMode.enDirectionPosAbs :=

enPosAbsDir;

arAxisCtrl_gb[Axis1.AxisNo].PosMode.Velocity :=

rVelocity_Axis1;

arAxisCtrl_gb[Axis1.AxisNo].PosMode.Acceleration :=

rAcceleration_Axis1;

arAxisCtrl_gb[Axis1.AxisNo].PosMode.Deceleration :=

rDeceleration_Axis1;

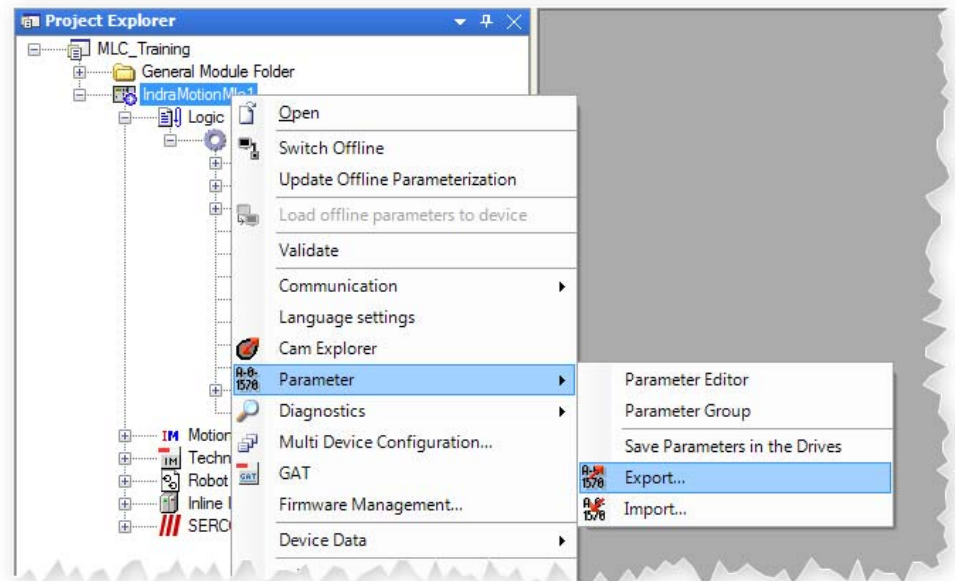
10. Check the correct function of your extension!

16 Parameter import and export

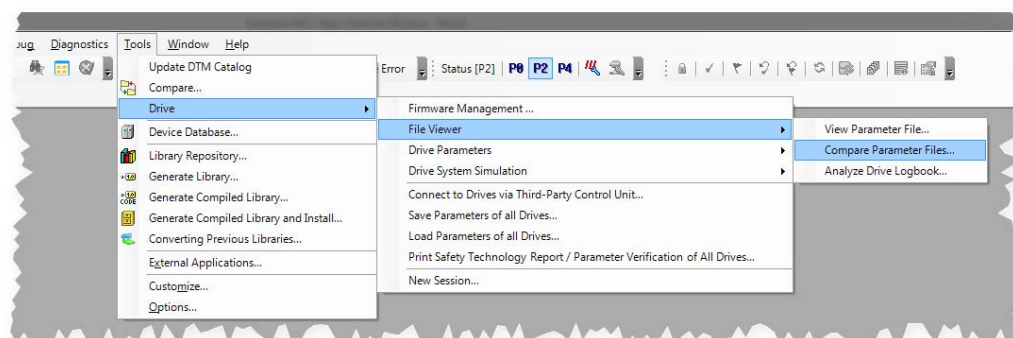
The parameters of the overall system are saved in a parameter file. This is sometimes helpful: A certain project state is saved and later it is always possible to return to this prior state.

16.1 Exercise

1. Export all parameters of the MLC and the drives:



2. Modify a parameter, e.g. A-0-0038 Torque/force limit value, bipolar
3. A direct comparison of saved parameters versus the online state is not supported. The parameters have to be exported again, using a different file name for the parameter file.
4. After a new export, a comparison based on the parameter files is supported:



17 Archiving and restoring projects

During this exercise the complete IndraWorks project is archived and restored later on.



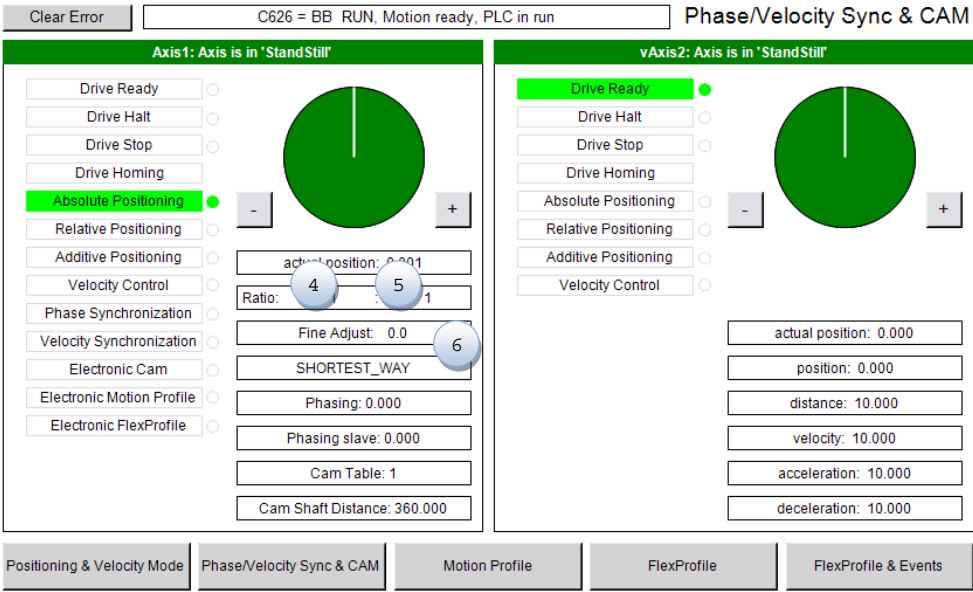
The archiving and restoring of IndraWorks projects is described in detail in the presentation. Please follow the instructions in the printout!

18 Velocity synchronization

In this exercise vAxis2 is used as master axis and Axis1 as slave axis. It is running in velocity synchronization.

18.1 Visualization

For this exercise the following visualization is used. It was added to the project already by the previous import.



Element #	Assigned Variable	Element #	Assigned Variable
1	uiRationN	3	rMasterFineAdjust
2	uiRationD		

18.2 Exercise

- Implement velocity synchronous operation for the real axis Axis1. The relevant components in the Axis Interface are as follows:

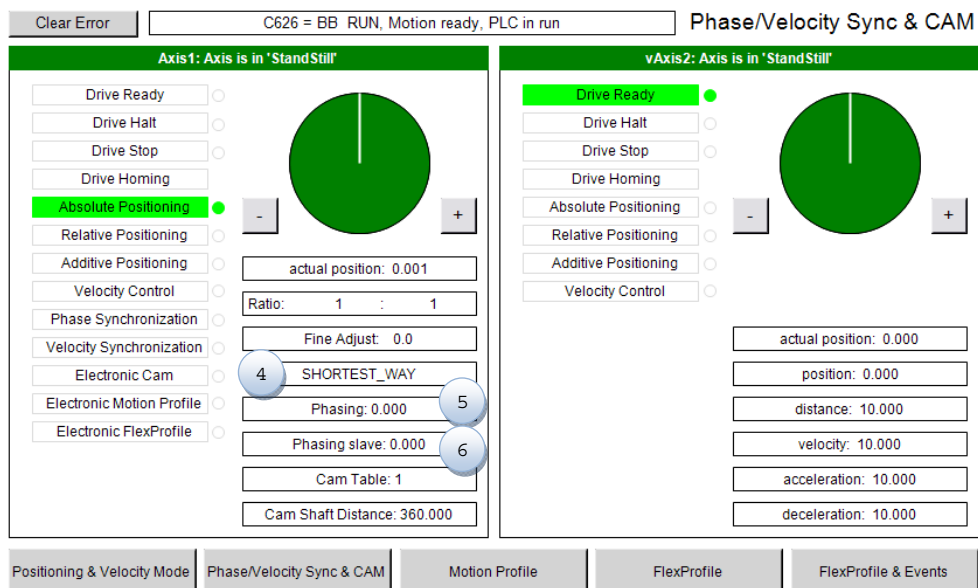
Element	Name	Type	Default	Cyclically scanned
arAxisCtrl_gb[]	SyncMode.OutputRevolution	UINT	1	Yes
	SyncMode.InputRevolution	UINT	1	Yes
	SyncMode.Fineadjust	REAL	0.0	Yes
	SyncMode.Master	AXIS_REF		Yes
	Admin.Axis	AXIS_REF		No
arAxisStatus_gb[]	Admin.MODE_SYNC_VEL	BOOL		Not applicable

19 Phase synchronization

In this exercise vAxis2 is used as master axis and Axis1 as slave axis. It is running in phase synchronization.

19.1 Visualization

For this exercise the visualization of the previous exercise is used.



Element #	Assigned Variable	Element #	Assigned Variable
1	enSyncMode	3	rPhaseOffset_PhasingSlave
2	rPhaseOffset_Phasing		

19.2 Exercise

- Implement phase synchronous operation for the real axis Axis1. The relevant components in the Axis Interface are as follows:

Element	Name	Type	Default	Cyclically scanned
arAxisCtrl_gb[]	SyncMode.OutputRevolution	UINT	1	Yes
	SyncMode.InputRevolution	UINT	1	Yes
	SyncMode.Fineadjust	REAL	0.0	Yes
	SyncMode.SyncDirection	MC_SYNC_DIRECTION	SHORTEST_WAY	Yes
	SyncMode.StartMode	MC_START_MODE	ABSOLUTE	No
	SyncMode.Master	AXIS_REF		Yes
	Admin.Axis	AXIS_REF		No
	SyncMode.PhaseOffset	REAL	0.0	Yes
	SyncMode.PhaseOffsetVel	REAL	1.0	No
	SyncMode.PhaseOffsetAcc	REAL	100.0	No
arAxisStatus_gb[]	Admin.MODE_SYNC_PHASE	BOOL		Not applicable
	SyncMode.PhasingSlaveDone	BOOL		Not applicable

20 Create drive CAM with CamBuilder

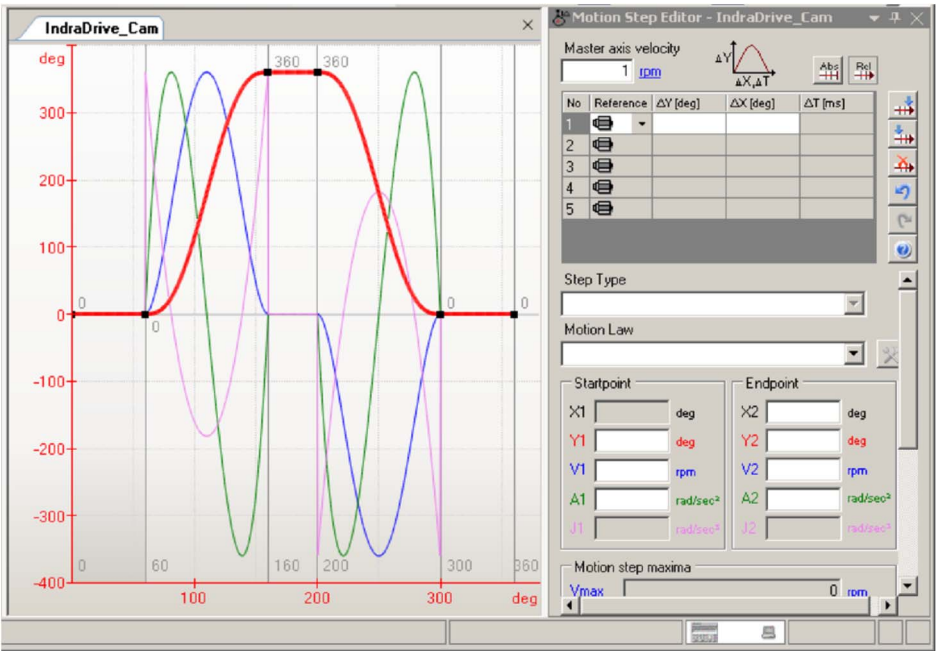
The CamBuilder is used to create a Drive CAM (point table on the drive).



The CamBuilder is an integral part of IndraWorks. You don't need to install any additional software to use it, but you have to activate it by entering a license code. On the training notebooks this has been done already.

20.1 Exercise

1. Add a CamPool to your project.
2. Add a new CAM to your CamPool and design it as follows:



No	Reference	ΔY [deg]	ΔX [deg]	ΔT [ms]
0		0	60	10000
1		360	100	16666.66
2		0	40	6666.666
3		-360	100	16666.66
4		0	60	10000

Motion Law

General Poly5

General Poly5

General Poly5

General Poly5

General Poly5

3. Download this CAM to the IndraDrive!

21 Electronic CAM (drive CAM)

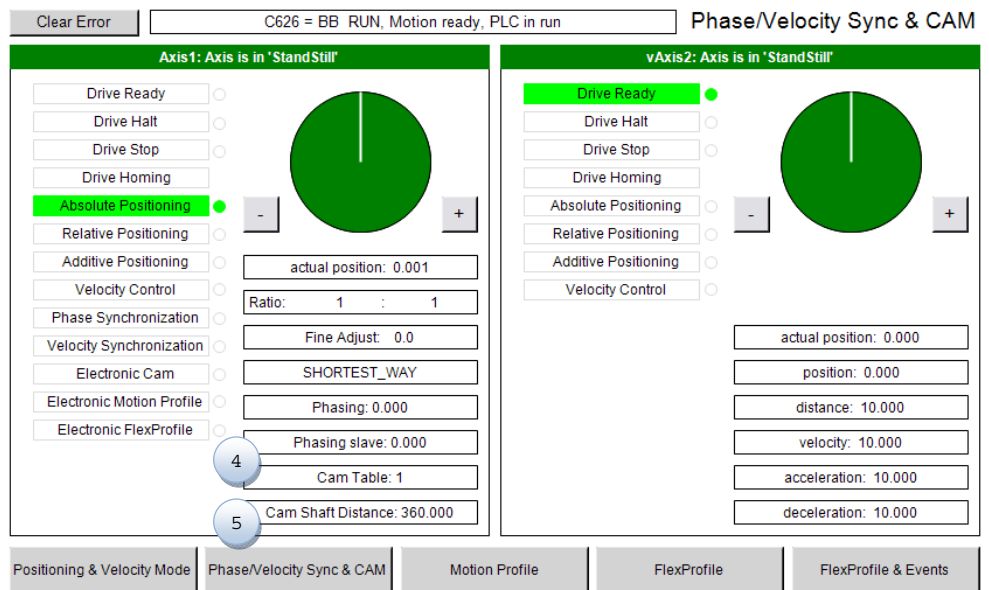
In this exercise vAxis2 is used as master axis and Axis1 as slave axis. Axis1 follows vAxis2 master using a drive CAM (point table).



In this exercise drive CAM's are used. As a prerequisite the functional package "Synchronization" has to be active on the drive and you need to use interpolation on the drive not interpolation on the control!

21.1 Visualization

For this exercise the visualization of the previous exercise is used.



Element #	Assigned Variable	Element #	Assigned Variable
1	uiCamId	2	rCamShaftDistance

21.2 Exercise

- Expand your motion program by the instructions which are required to run the drive CAM on Axis1. The relevant components in the Axis Interface are as follows:

Element	Name	Type	Default	Cyclically scanned
arAxisCtrl_gb[]	SyncMode.OutputRevolution	UINT	1	Yes
	SyncMode.InputRevolution	UINT	1	Yes
	SyncMode.Fineadjust	REAL	0.0	Yes
	SyncMode.CamShaftDistance	REAL	360.0	Yes
	SyncMode.SyncDirection	MC_SYNC_DIRECTION	SHORTEST_WAY	Yes
	SyncMode.StartMode	MC_START_MODE	ABSOLUTE	No
	SyncMode.CamTableID	MC_CAM_ID	CAM_TABLE_1	Yes
	SyncMode.Master	AXIS_REF		Yes
	Admin.Axis	AXIS_REF		No
	SyncMode.PhaseOffset	REAL	0.0	Yes
	SyncMode.PhaseOffsetVel	REAL	1.0	No
	SyncMode.PhaseOffsetAcc	REAL	100.0	No
arAxisStatus_gb[]	SyncMode.PhasingSlaveDone	BOOL		
	SyncMode.CamActiveID	MC_CAM_ID		
	Admin.MODE_SYNC_CAM	BOOL		Not applicable

22 Motion Profile

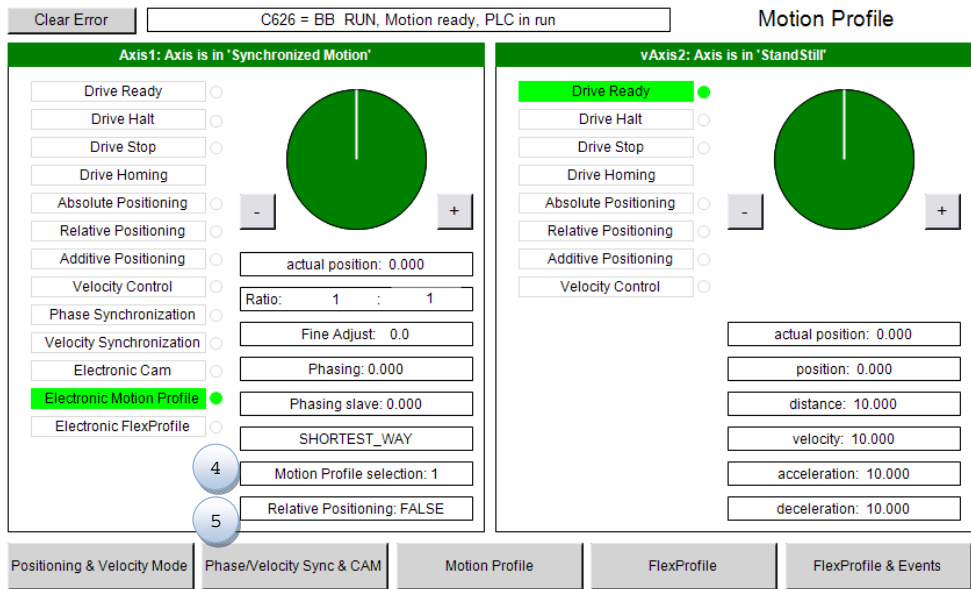
In this exercise vAxis2 is used as master axis and Axis1 as slave axis. The real axis follows the master using a control CAM (Motion Profile).



In this exercise Motion Profiles are used. As a prerequisite interpolation on the control has to be selected for the slave axis!

22.1 Visualization

For this exercise the following visualization is used. It was added to the project already by the previous import.



Element #	Assigned Variable	Element #	Assigned Variable
1	uiMotionProfileSet	2	bRelativePositioning

22.2 Exercise

1. The existing project has to be modified:
 - a. Delete Axis1 from the project.
 - b. Select an IndraDrive from the device library and add it to the project
 - c. Be sure to select the appropriate firmware version of the drive
 - d. For the new drive uncheck "Interpolation on the drive"
2. Create the following Motion Profile:



3. Download the Motion Profile to Axis1
4. Expand your motion program to run the Motion Profile on Axis1. The relevant components in the Axis Interface are as follows:

Element	Name	Type	Default	Cyclically scanned
arAxisCtrl_gb[]	SyncMode.OutputRevolution	UINT	1	Yes
	SyncMode.InputRevolution	UINT	1	Yes
	SyncMode.Fineadjust	REAL	0.0	Yes
	SyncMode.Profile.SetSelection	UINT	0	Yes
	SyncMode.Profile.RelativePositioning	BOOL	FALSE	No
	SyncMode.SyncDirection	MC_SYNC_DIRECTION	SHORTEST_WAY	Yes
	SyncMode.StartMode	MC_START_MODE	ABSOLUTE	No
	SyncMode.Master	AXIS_REF		Yes
	Admin.Axis	AXIS_REF		No
	SyncMode.PhaseOffset	REAL	0.0	Yes
	SyncMode.PhaseOffsetVel	REAL	1.0	No
	SyncMode.PhaseOffsetAcc	REAL	100.0	No
arAxisStatus_gb[]	SyncMode.PhasingSlaveDone	BOOL		Not applicable
	SyncMode.ProfileActiveSet	UINT		Not applicable
	Admin.MODE_SYNC_CAM	BOOL		Not applicable

5. Use the IndraLogic Trace function to check the correct operation.

23 Create FlexProfile with CamBuilder

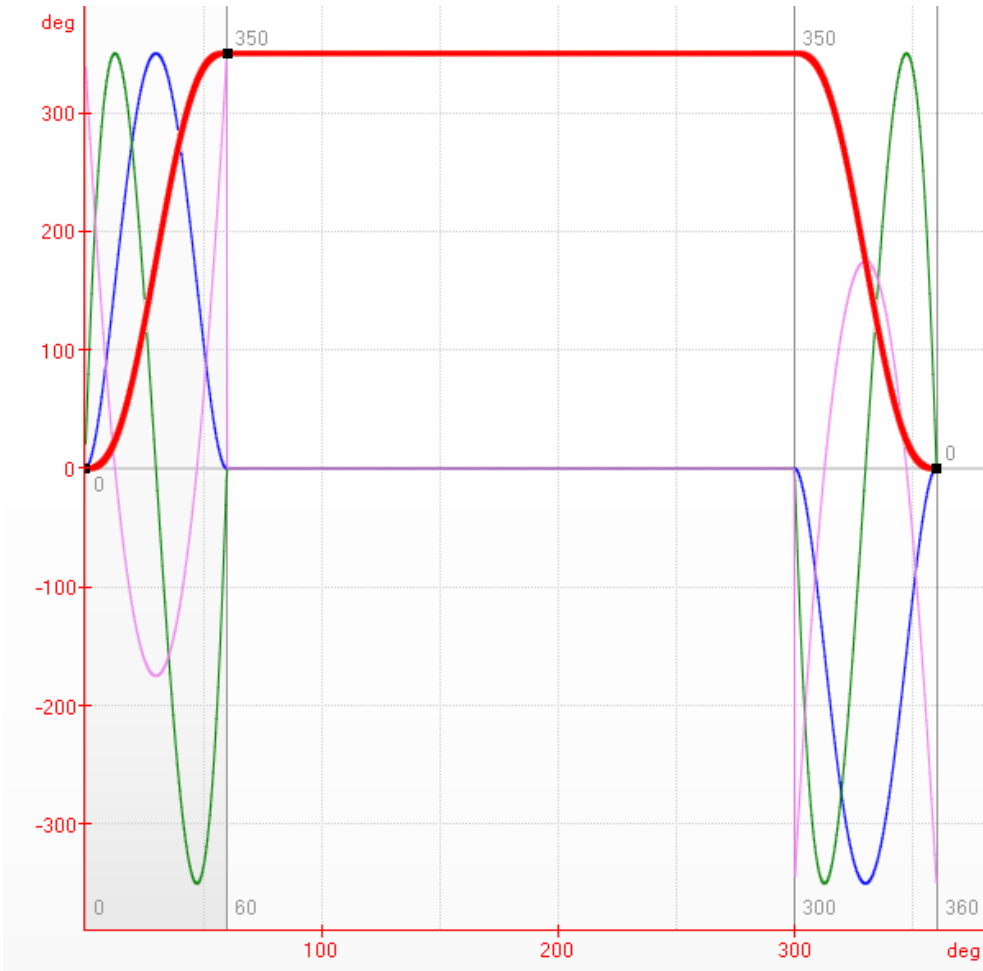
In this exercise a FlexProfile is created and downloaded to the MLC.



A prerequisite for the use of the FlexProfile is that the corresponding axis is configured with “Interpolation on the control”. Otherwise a download of the FlexProfile is not possible.

23.1 Exercise

► Create the following FlexProfile (master axis velocity: 20rpm) and download it:



No	Reference	ΔY [deg]	ΔX [deg]	ΔT [ms]
0		350	60	500
1		0	240	2000
2		-350	60	500

Motion Law

R → R Poly5

R → R Standstill

FlexStep Poly5

24 FlexProfile

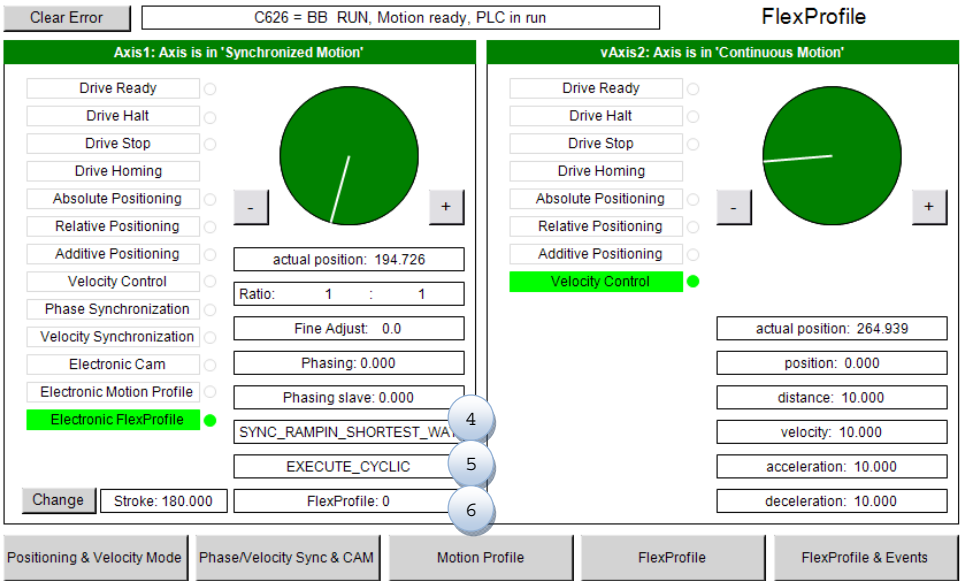
In this exercise vAxis2 is used as master axis and Axis1 as slave axis. The real axis follows the master using a FlexProfile.



In this exercise FlexProfiles are used. As a prerequisite interpolation on the control has to be selected for the slave axis!

24.1 Visualization

For this exercise the following visualization is used. It was added to the project already by the previous import.



Element #	Assigned Variable	Element #	Assigned Variable
1	enSyncType	3	uiFlexProfileSet
2	enExecutionMode		

24.2 Exercise

1. Expand your motion program to run the FlexProfile on Axis1. The relevant components in the Axis Interface are as follows:

Element	Name	Type	Default	Cyclically scanned
arAxisCtrl_gb[]	SyncMode.OutputRevolution	UINT	1	Yes
	SyncMode.InputRevolution	UINT	1	Yes
	SyncMode.Fineadjust	REAL	0.0	Yes
	SyncMode.Profile.SetSelection	UINT	0	Yes
	SyncMode.Profile.UseSwitchingPosition	BOOL	FALSE	No
	SyncMode.Profile.SwitchingPosition	REAL	0.0	No
	SyncMode.Profile.ProfileEntry	MB_PROFILE_START_POINT	SLAVE_ORIGIN_MASTER_ORIGIN	No
	SyncMode.Profile.MasterOffset	REAL	0.0	No
	SyncMode.Profile.SlaveOffset	REAL	0.0	No
	SyncMode.Profile.SyncType	MB_SYNC_TYPE	SYNC_RAMPIN_SHORTEST_WAY	No
	SyncMode.Profile.SyncVelocity	REAL	10.0	No
	SyncMode.Profile.SyncAcceleration	REAL	10.0	No
	SyncMode.Profile.ExecutionMode	MB_EXECUTION_MODE	EXECUTE_CYCLIC	No
	SyncMode.Master	AXIS_REF		Yes
	Admin.Axis	AXIS_REF		No
	SyncMode.PhaseOffset	REAL	0.0	Yes
	SyncMode.PhaseOffsetVel	REAL	1.0	No
	SyncMode.PhaseOffsetAcc	REAL	100.0	No
arAxisStatus_gb[]	SyncMode.PhasingSlaveDone	BOOL		Not applicable
	SyncMode.ProfileActiveSet	UINT		Not applicable
	Admin.MODE_FLEX_PROFILE	BOOL		Not applicable

2. Use the IndraLogic Trace function to check the correct operation.

25 Online creation of FlexProfiles

This exercise demonstrates how to create FlexProfiles online from the motion program.

25.1 Visualization

For this exercise the following visualization is used. It was added to the project already by the previous import.

Clear Error C626 = BB RUN, Motion ready, PLC in run FlexProfile

Axis1: Axis is in 'Synchronized Motion'

Drive Ready ☐

Drive Halt ☐

Drive Stop ☐

Drive Homing ☐

Absolute Positioning ☐

Relative Positioning ☐

Additive Positioning ☐

Velocity Control ☐

Phase Synchronization ☐

Velocity Synchronization ☐

Electronic Cam ☐

Electronic Motion Profile ☐

Electronic FlexProfile ☒

4 Change 5 Stroke: 180.000 6 EXECUTE_CYCLIC

actual position: 194.726

Ratio: 1 : 1

Fine Adjust: 0.0

Phasing: 0.000

Phasing slave: 0.000

SYNC_RAMPIN_SHORTEST_WAY

FlexProfile: 0

vAxis2: Axis is in 'Continuous Motion'

Drive Ready ☐

Drive Halt ☐

Drive Stop ☐

Drive Homing ☐

Absolute Positioning ☐

Relative Positioning ☐

Additive Positioning ☐

Velocity Control ☒

actual position: 264.939

position: 0.000

distance: 10.000

velocity: 10.000

acceleration: 10.000

deceleration: 10.000

Positioning & Velocity Mode Phase/Velocity Sync & CAM Motion Profile FlexProfile FlexProfile & Events

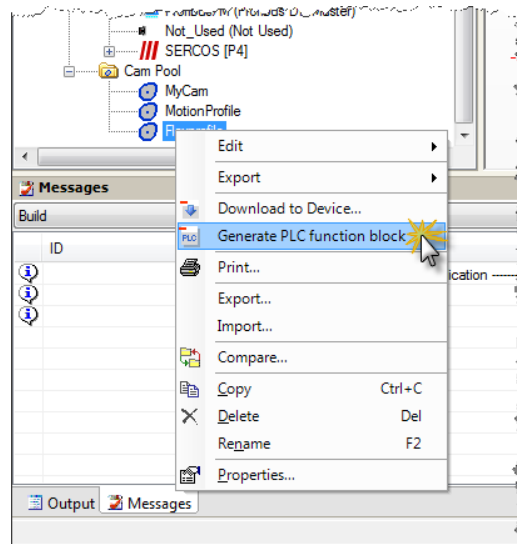
Element #	Assigned Variable	Element #	Assigned Variable
1	bExeChangeFlexProfileSet	3	bDoneChangeFlexProfile
2	rStrokeFlexProfile		

25.2 Exercise

1. The stroke shall be variable, use the Formula Editor to change it:

No	Reference	ΔY [deg]	ΔX [deg]	ΔT [ms]
0		f 350	60	500
1		0	240	2000
2		-350	60	500

2. Enter “**=stroke**” in the column ΔY of the first step.
3. Generate a function block for this FlexProfile:



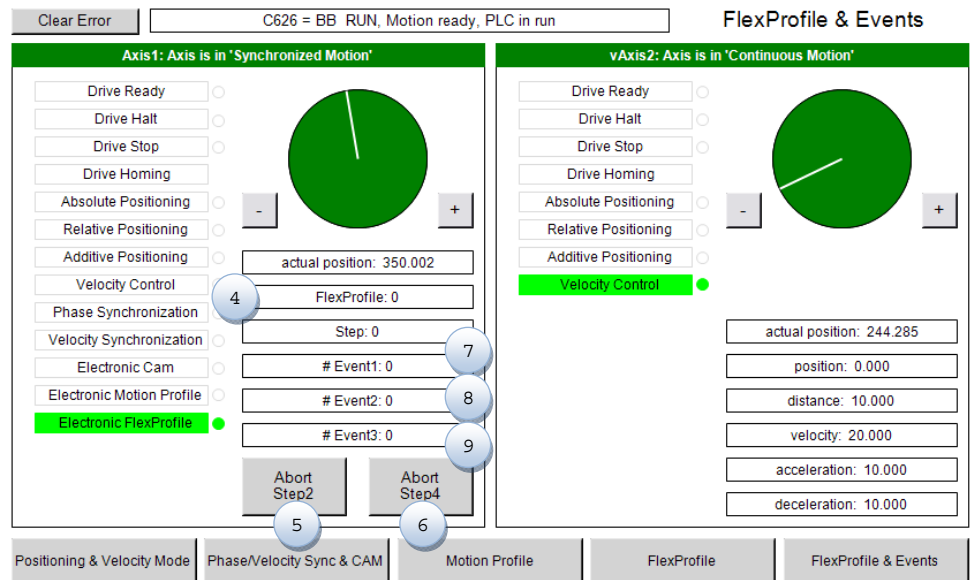
4. As a result this FlexProfile can be created dynamically from the Motion Program by calling the generated function block. The variable **stroke** is an input parameter of the function block.
5. Add an instance of this function block to your program.
6. Check the proper operation!

26 FlexProfile and Events

Before using Events, a new FlexProfile is created.

26.1 Visualization

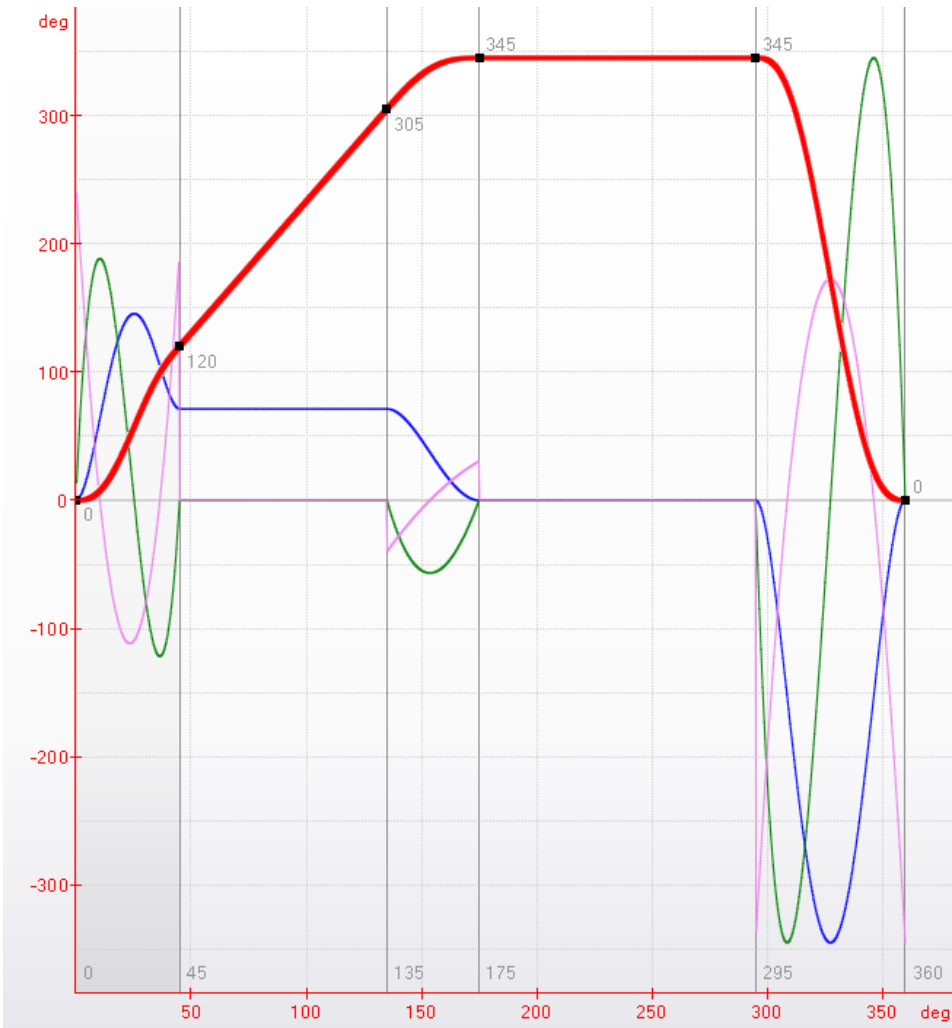
For this exercise the following visualization is used. It was added to the project already by a previous import.



Element #	Assigned Variable	Element #	Assigned Variable
1	uiActFlexStep	4	uiCounterEvent1
2	bAbortStep2	5	uiCounterEvent2
3	bAbortStep4	6	uiCounterEvent3

26.2 Exercise

1. Create the following FlexProfile (master axis velocity: 20rpm) and download it:



No	Reference	ΔY [deg]	ΔX [deg]	ΔT [ms]
0		120	45	375
1		185	90	750
2		40	40	333.3333
3		0	120	1000
4		-345	65	541.6667

Motion Law

R → V Poly5

V → V Const. velocity

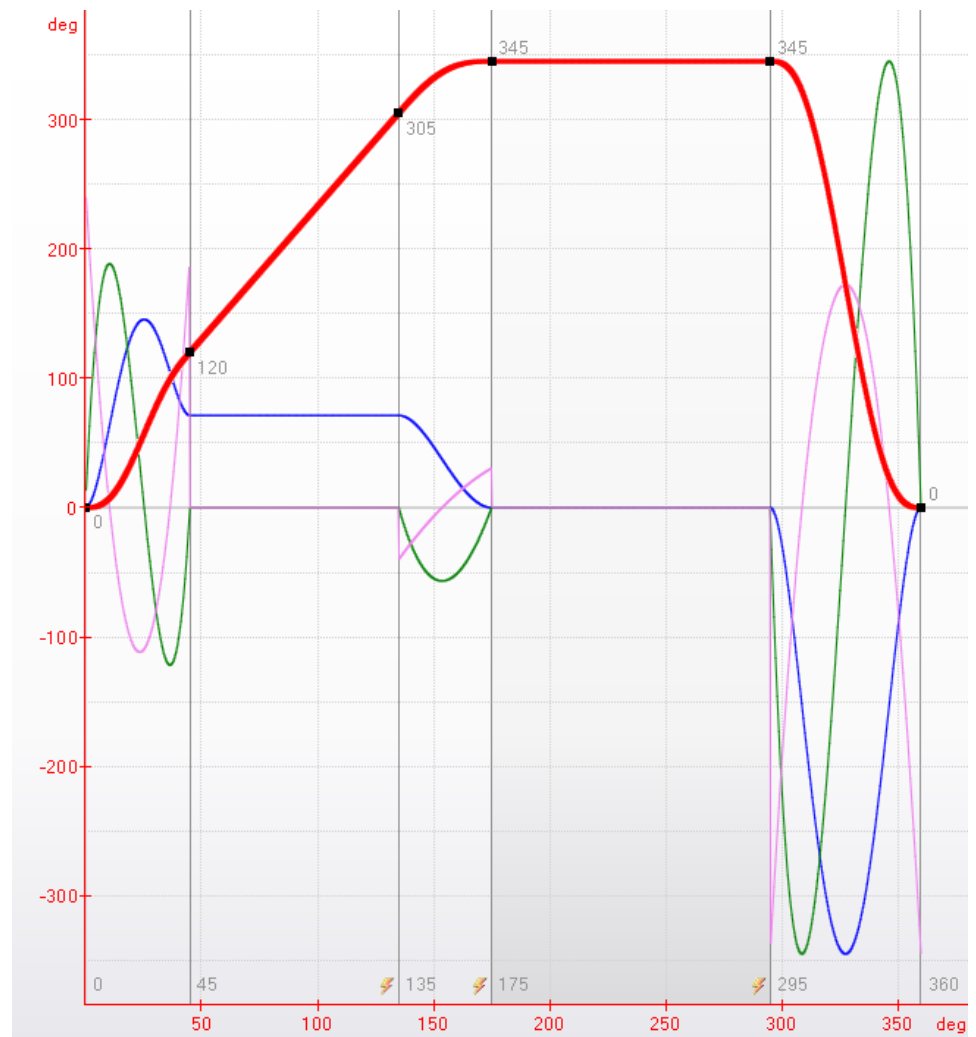
V → R Poly5

R → R Standstill

FlexStep

2. Test the FlexProfile by running it on Axis1, use vAxis2 as master axis.

3. Modify the FlexProfile as follows:



Set Master axis velocity to 8 rpm and the duration of the 4th step to 3000ms!

4. Configure 3 events for the FlexProfile and download it to Axis1:

No.	Motion Step	Trigger	Value	Unit	Bit	ActionMode
0	Step 1	PLC Signal	0	--	Bit 0	Set Bit immediately and immediately connect next motion step
1	Step 2	Relative slave distance mov...	15	deg	Bit 1	Set Bit immediately
2	Step 3	PLC Signal	0	--	Bit 2	Set Bit immediately and immediately connect next motion step

5. Create a high prior task and count the events.
6. Display the actual FlexProfile step number.
7. Use the two buttons “Abort Step2” and “Abort Step 4” to fire the events declared with a PLC signal.

27 Notes

[illegible]

Bosch Rexroth
Training Center Boxtel
Kruisbroeksestraat 1
5281 RV Boxtel

Tel. +031 411 651 416
Fax +031 411 651 483
nltraining@boschrexroth.nl
www.boschrexroth.nl/Training